

Comprehensive Design Report: Safe Semantic Planner

PCCST503 - Machine Learning

1 Introduction and Objectives

This report outlines the detailed design and implementation of a safe semantic planner operating within a finite Cartesian state space, denoted as $S = \{s_1, s_2, \dots, s_n\}$ embedded in $\mathbb{R}^d$. The primary optimization objectives are to reliably reach the goal state, absolutely avoid predefined bad states $B = \{b_1, b_2, \dots, b_k\}$, minimize the cumulative transition cost, and maximize the minimum Euclidean clearance distance to any bad state during traversal. To satisfy the requirement for efficient operation in dynamic environments where transition availability may change, the Lifelong Planning A* (LPA*) algorithm was selected as the core search mechanism.

2 Algorithmic Foundation: LPA*

The planner uses LPA* to support incremental replanning without fully rebuilding the search tree when the environment changes. The algorithm maintains two estimates of the start-distance for each state: a g -value (the current shortest path cost) and an rhs -value (one-step lookahead based on predecessors). Inconsistencies between g and rhs dictate which states require expansion.

3 System Architecture and Implementation

3.1 State and Transition Representation

States are defined by a unique 64-bit unsigned integer identifier (`uint64_t`) and an embedding vector of double-precision floats representing coordinates in $\mathbb{R}^d$. Transitions are modeled as directed edges containing an identifier, source state, destination state, baseline cost, safety score, reliability metric, and a boolean availability flag.

3.2 Data Structures

The implementation relies heavily on hash tables to achieve $O(1)$ average-case state lookups:

- **State Maps:** `std::unordered_map` structures track the g -values, rhs -values, and full state objects mapped by their `uint64_t` IDs.
- **Graph Topology:** Forward (`adj`) and backward (`pred`) adjacency lists are stored using `unordered_map` paired with `std::vector` to allow rapid traversal of neighbors during vertex updates.
- **Priority Queue:** The open list U is implemented via a `std::set` storing nested pairs: `pair<pair<double, double>, uint64_t>`. This automatically sorts states lexicographically by their two-part heuristic keys (k_1, k_2) .
- **Bad States:** Stored in a `std::unordered_set` (`badSet`) for $O(1)$ membership checking to instantly prune transitions directly connected to hazard zones.

3.3 Heuristic Function

The heuristic function $h(u, v)$ estimates the remaining cost to the goal using the standard Euclidean distance between the multidimensional embeddings of two states. For states u and v with dimensions d :

$$h(u, v) = \sqrt{\sum_{i=1}^d (u_i - v_i)^2}$$

This function is consistent and admissible, ensuring the underlying A* expansion remains optimal.

3.4 Safety Computation and Penalties

To satisfy the optimization objective of maximizing distance from bad states, the planner computes a dynamic safety penalty for each evaluated node. The minimum distance $D_{min}(u)$ from a state u to any bad state $b \in B$ is calculated using the heuristic distance function. The safety penalty $P(u)$ added to the edge weight is computed as:

$$P(u) = \frac{1}{D_{min}(u) + 0.1}$$

If $D_{min}(u) = 0$, an extreme penalty of 10000.0 is applied, effectively blocking the path. The total effective weight of traversing from u to v becomes $cost(u, v) + P(v)$.

4 Complexity Analysis

- **Time Complexity:** The initial planning phase executes in $O(E \log V)$ time, where V is the number of states and E is the number of transitions. The priority queue insertions and extractions dominate the computation. However, during dynamic replanning (e.g., an edge removal), LPA* only updates the locally inconsistent vertices, drastically reducing the effective time complexity compared to a naive A* restart.
- **Space Complexity:** The overall space complexity is strictly bounded by $O(V + E)$. Storing the graph topology requires $O(E)$ space across the `adj` and `pred` maps, while the g -values, rhs -values, and state embeddings consume $O(V)$ space.

5 Handling Dynamic Environments

The system gracefully handles periodic changes such as transition removals. When an edge's `available` flag is toggled to false, the system does not purge the g and rhs maps. Instead, it updates the affected vertices using `updateVertex(u)`, which recalculates the rhs -value based purely on currently available incoming edges. Inconsistencies are pushed to the priority queue, allowing `computeShortestPath()` to patch the existing solution space optimally.